

NexusFinance — Security & Compliance

Digital Lending Platform for Cambodia

EN

Document Number	NF-DOC-003
Version	1.0
Date	August 2026
Prepared By	NDX Digital Support
Classification	Internal & Confidential

Security • Auth • OTP • Data Protection • Audit

Table of Contents

1. Authentication Architecture
2. Password Security
3. OTP Verification System
4. Session & Token Management
5. Role-Based Access Control
6. Audit Logging
7. API Security
8. Data Protection
9. Payment Security

1. Authentication Architecture

NexusFinance uses a multi-layer authentication system combining email/password with OTP verification. All authentication flows are handled by the Express backend and validated against the Supabase PostgreSQL database.

1.1 Auth Flow Summary

Flow	Steps	Security Measure
Register	Email + password → OTP email → verify → account created	OTP hash, 10-min expiry
Login	Email + password → bcrypt verify → JWT issued	Rate limited, bcrypt compare
Forgot Password	Email → OTP → verify → set new password	OTP verified before change
Change Password	OTP email → verify → update password	Requires active session + OTP

2. Password Security

- Passwords are hashed using bcrypt with salt rounds = 10
- Plain-text passwords are never stored or logged
- Password comparison uses bcrypt.compare() — constant-time to prevent timing attacks
- Minimum password length: 6 characters (enforced on registration)
- All password changes require OTP verification first

3. OTP Verification System

Property	Value
Code Length	6 digits
Hash Method	bcrypt (10 rounds)
Expiry	10 minutes from generation
Verified Window	15 minutes after successful verification
Storage	otp_code (hashed), otp_expires_at, otp_verified_at columns on

	nexus_users
Delivery	Brevo transactional email API
Resend Limit	300 seconds cooldown

Security note

OTP codes are stored as bcrypt hashes, not plain text. Even if the database is compromised, the codes cannot be reversed.

4. Session & Token Management

- JWT tokens are issued on successful login
- Token payload: { id, email, name, role, iat }
- Tokens are validated server-side on every API request
- No refresh tokens — re-login required after expiry
- Tokens are stored in browser localStorage (client-side)

5. Role-Based Access Control

Role	Permissions
customer	Apply for loans, view own transactions, manage profile, make payments
loan-officer	Review applications, approve/reject loans, view assigned cases
super-admin	Full access: analytics, user management, config, audit logs, bot commands

All API endpoints enforce role checks server-side. A user cannot access another role's endpoints even with a valid JWT token.

6. Audit Logging

- Every sensitive action is logged to the nexus_audit_logs table
- Log entry: timestamp, action type, description, user metadata (id, email, name, role)
- Actions logged: login, register, loan create, loan update, profile update, password change
- Audit logs are append-only and viewable by Super Admin

7. API Security

- Rate limiting on auth endpoints (login, register, OTP, password reset)
- CORS configured to allow only the frontend origin
- Environment variables for all secrets (JWT_SECRET, API keys, database credentials)
- No secrets committed to the Git repository
- HTTPS enforced on all deployed endpoints

8. Data Protection

- User data stored in Supabase (AWS-hosted PostgreSQL)
- Service role key used for server-side operations (bypasses RLS)
- Frontend uses anon key with RLS policies
- No personally identifiable information in logs or error messages
- OTP codes never stored in plain text

9. Payment Security

- KHQR codes are generated server-side with CRC16 verification
- Payment verification requires server-side API calls (not client-side)
- ABA PayWay webhooks use HMAC signature verification
- All payment transactions are logged in the audit trail
- Sandbox mode active — no real money processed yet